Learning Go's net/http Package

A complete study guide from our conversation

Topics covered

- How a Go HTTP server listens on a port
- What happens when a client sends a request
- Handlers, routers, and responses
- The difference between http.Handler and http.HandlerFunc
- How http.ServeMux and HandleFunc work together
- Working code, common misconceptions, and practice exercises

Designed for beginner-friendly learning and for building a Go backend API.

Reference article: A Complete Beginner's Guide to Go's net/http Package

1. The big picture: how an HTTP request travels

A backend server must be running before a browser, Postman, or frontend can send it a request. Go's standard-library package `net/http` provides tools for accepting HTTP connections, routing requests, and writing responses.

Stage	What happens
1. Client	A browser, Postman, or frontend sends an HTTP request, such as GET /jobs.
2. Operating system / listener	Network traffic is delivered to the application listening on the target address and port.
3. HTTP server	Go accepts connections and parses incoming HTTP requests.
4. Router (ServeMux)	The router matches the request against registered patterns.
5. Handler	The selected handler performs the application's work and writes a response.
6. Response	The client receives an HTTP status, headers, and a response body.

A simple mental model

Server: receives and serves HTTP requests. **Router:** selects a handler. **Handler:** performs work for a request. **ResponseWriter:** lets the handler write the response.

For a job portal, examples might be GET /jobs to list jobs, POST /jobs to create a job, and GET /health to check whether the API responds.

2. What is a port?

A port is a number used by the operating system to direct network traffic to a listening application or service. A computer can run several network services on different ports.

Example address/port	Possible local development use
localhost:3000	Next.js frontend
localhost:8080	Go backend
localhost:5432	PostgreSQL
localhost:443	Common HTTPS port

Important: a port number does not create a server by itself. The Go program must ask the operating system to listen on the address.

Understanding an address

```
http://localhost:8080
```

- **http://** — use the HTTP protocol.
- **localhost** — the local computer.
- **8080** — the port to connect to.

Common ListenAndServe address forms include `":8080"`, `"localhost:8080"`, and `"127.0.0.1:8080"`. The first generally listens on available local network interfaces; the latter two target the local machine, with 127.0.0.1 explicitly selecting IPv4 loopback.

3. Start your first Go HTTP server

```
package main

import "net/http"

func main() {
    http.ListenAndServe(":8080", nil)
}
```

Run it with `go run main.go`. The terminal may appear idle because the server is waiting for requests. Visit `http://localhost:8080` and you will typically get `404 page not found`.

> That 404 is expected: the server is running, but no route has been registered to handle the requested path. Accepting requests and having an application route for a URL are separate things.

What does ListenAndServe do?

```
http.ListenAndServe(address, handler)
```

Argument	Meaning
address	Network address, including the port, such as :8080.
handler	The HTTP handler that receives requests. If nil, Go uses http.DefaultServeMux.

`ListenAndServe` listens on a TCP address and serves HTTP requests through the supplied handler. It normally blocks while the server is running, so code after the call usually won't run until it returns.

Handle errors

```
package main

import (
    "log"
    "net/http"
)

func main() {
    log.Println("Starting server on port 8080...")

    err := http.ListenAndServe("127.0.0.1:8080", nil)
    if err != nil {
        log.Fatal(err)
    }
}
```

If another process is already listening on the same address and port, the second server will usually fail to start with a bind/address-in-use error. Press Ctrl+C in the terminal to stop a server during local practice.

4. Create a handler and a route

```
package main

import (
    "fmt"
    "log"
    "net/http"
)

func homeHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Hello from my Go server!")
}

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/", homeHandler)

    log.Fatal(http.ListenAndServe("127.0.0.1:8080", mux))
}
```

Visit `http://127.0.0.1:8080/`. The response body should be `Hello from my Go server!`.

- `w http.ResponseWriter` lets the handler write the HTTP response.
- `r *http.Request` contains details about the incoming request, such as its method, URL, headers, and body.
- `fmt.Fprintln(w, ...)` writes to the HTTP response, not to the terminal.

> If a handler writes a response body without first setting a status, Go normally commits a 200 OK status.

5. What is http.Handler?

An interface describes behavior. The `http.Handler` interface requires a type to provide a `ServeHTTP` method with the correct signature.

```
type Handler interface {
    ServeHTTP(ResponseWriter, *Request)
}
```

The actual types belong to the `http` package. Any type with the matching method satisfies the interface automatically; Go does not require an explicit 'implements' declaration.

Example: a struct that implements Handler

```
package main

import (
    "fmt"
    "net/http"
)

type HomeHandler struct{}

func (h HomeHandler) ServeHTTP(
    w http.ResponseWriter,
    r *http.Request,
) {
    fmt.Fprintln(w, "Welcome to my Go server!")
}

func main() {
    handler := HomeHandler{}
    http.ListenAndServe("127.0.0.1:8080", handler)
}
```

Why does `HomeHandler{}` work as the second argument? Because its type has the required `ServeHTTP` method and therefore satisfies `http.Handler`.

A struct does not automatically implement every interface. It must have all the required methods with matching signatures.

6. What is http.HandlerFunc?

You may prefer to write a normal function rather than define a struct and a method:

```
func homeHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Hello!")
}
```

A normal function with this signature does not itself have a `ServeHTTP` method. `http.HandlerFunc` solves this by being a named function type that already has the required method.

Simplified illustration

```
type HandlerFunc func(http.ResponseWriter, *http.Request)

func (f HandlerFunc) ServeHTTP(
    w http.ResponseWriter,
    r *http.Request,
) {
    f(w, r)
}
```

This is an explanatory version of the standard-library idea. The method calls the original function with the same response writer and request.

Convert a function to HandlerFunc

```
var handler http.Handler = http.HandlerFunc(homeHandler)
```

Read it in three stages:

- `homeHandler` is the ordinary function you wrote.
- `http.HandlerFunc(homeHandler)` converts it to the named function type.
- The converted value satisfies `http.Handler` because HandlerFunc has a `ServeHTTP` method.

Key correction: HandlerFunc does not inspect or test the function to see if it qualifies. The function signature must be compatible at compile time, and the HandlerFunc type supplies the required method.

7. What is http.ServeMux?

A server commonly needs many routes. `http.ServeMux` is Go's standard HTTP request multiplexer (router). It registers patterns and directs matching requests to their handlers.

```
package main

import (
    "fmt"
    "log"
    "net/http"
)

func homeHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Welcome to the Job Portal!")
}

func jobsHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Here are the available jobs.")
```

```
}

func healthHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "API is running!")
}

func main() {
    mux := http.NewServeMux()

    mux.HandleFunc("/", homeHandler)
    mux.HandleFunc("/jobs", jobsHandler)
    mux.HandleFunc("/health", healthHandler)

    log.Fatal(http.ListenAndServe("127.0.0.1:8080", mux))
}
```

Try these URLs in a browser or Postman:

- `http://127.0.0.1:8080/` — homepage handler.
- `http://127.0.0.1:8080/jobs` — jobs handler.
- `http://127.0.0.1:8080/health` — health handler.

Handle versus HandleFunc

Method	What it accepts	Example
mux.Handle	A value implementing http.Handler	mux.Handle("/jobs", handler)
mux.HandleFunc	A function with signature func(ResponseWriter, *Request)	mux.HandleFunc("/jobs", jobsHandler)

`HandleFunc` does not create the function itself. You write the function first, then register it against a route.

In modern Go, a pattern such as `"/jobs"` matches that exact path; `"/"` is a catch-all pattern. More specific patterns and method-aware patterns are available in modern ServeMux versions.

8. How the pieces work together

Component	Main job
http.ListenAndServe	Listens for network connections and serves HTTP requests.
http.ServeMux	Matches incoming requests to registered patterns.
http.Handler	Defines the required ServeHTTP behavior.
http.HandlerFunc	Adapts a compatible function to the Handler interface.
mux.HandleFunc	Registers a function for a route, adapting it internally.
Your handler function	Performs the request's application logic and writes the response.

For example, when Postman sends GET /users, the server receives the request, passes it to the mux, the mux selects the registered /users handler, and that handler writes the response.

> The router does not execute every handler. It selects the matching handler for the request.

9. The learner's original understanding — refined

The original explanation was largely correct: start a server on a port, register several handlers, and use a router to choose which handler should run. The refinements are:

- Use `mux.HandleFunc("/routeName", handlerFunction)` to register a function. `http.Handler` is an interface, not a route-registration function.
- The mux's `HandleFunc` method registers a function that already exists; it doesn't create that function.
- `http.HandlerFunc` adapts a function to the Handler interface. It doesn't perform a runtime qualification check.
- The server accepts requests and delegates to the handler you pass it. Passing the mux means the mux handles routing.
- `ListenAndServe` normally blocks while serving; it is often the last statement in main, but the important behavior is that it keeps serving until it returns.

10. Practice exercises

Concept checks

1. What typically happens when you start `http.ListenAndServe(":8080", nil)` without registering any routes and request /?

Answer: The server runs, but the default mux normally returns 404 for an unmatched route.

2. What does 8080 represent in `":8080"`?

Answer: The TCP port number.

3. Why does code after ListenAndServe normally not execute?

Answer: ListenAndServe blocks while it serves requests and returns only when serving ends or an error occurs.

4. What happens if a second server tries to bind to the same address and port?

Answer: It will usually fail to bind while the first listener is active.

5. Why does HomeHandler{} satisfy http.Handler?

Answer: Its type implements the required ServeHTTP method.

6. What does http.HandlerFunc(homeHandler) do?

Answer: It converts the compatible function to the HandlerFunc type, which has a ServeHTTP method.

7. What is ServeMux responsible for?

Answer: Matching incoming requests to registered handlers.

Coding challenge

Extend the multi-route server with a new route:

- Path: `/about`
- Handler function: `aboutHandler`
- Response body: `This is my first Go API.`
- Register it using `mux.HandleFunc` and test it in Postman.

> Hint: write the function first, register the route, and keep the existing ListenAndServe call.

11. A compact reference

```
mux := http.NewServeMux()

mux.HandleFunc("/", homeHandler) // register a function
mux.Handle("/custom", customHandler) // register an http.Handler

err := http.ListenAndServe(":8080", mux)
if err != nil {
    log.Fatal(err)
}
```

Remember the roles: server = listens and serves; mux = routes; handler = does the work; Handler = interface contract; HandlerFunc = function adapter.

12. Further reading

Go standard library documentation:

- net/http package — server, handlers, request/response types, and routing.

Original learning article:

A Complete Beginner's Guide to Go's net/http Package

Suggested learning order: server and ports → request and response → Handler and HandlerFunc → ServeMux and routing → JSON decoding and validation → middleware → PostgreSQL integration and CRUD endpoints.